

Audio Processing & Speech AI: Teaching Machines to Listen

KAUST–Mawhiba IOAI Training

Week 4 · Day 20



جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

Part 1: Audio Fundamentals

1. Bridge & Motivation
2. Sound Fundamentals
3. Spectrograms
4. Mel-Scale & MFCCs
5. ASR Evolution

Part 2: Modern Audio Models

6. Self-Supervised Audio (HuBERT)
7. Whisper: Speech Recognition
8. Multimodal Audio Models
9. Audio Tasks & Transfer Learning
10. Day 20 Summary

- ▶ **Day 17:** Built the *Transformer* from scratch — attention, encoder blocks
- ▶ **Day 18:** Met *BERT* and *GPT* — pre-trained models for text
- ▶ **Day 19:** Used *LLMs*, saw *Vision Transformers*, learned *LoRA*
- ▶ We've taught AI to **read** (text) and **see** (images)
- ▶ **Today's question:** Can AI *listen*?

Running Example

Throughout today, we will follow one audio clip:

A recording of someone saying "Hello, how are you?"

We will trace it from raw sound waves all the way to text transcription!

- ▶ Think about how many times you used audio AI *today*:
- ▶ **Voice assistants:** “Hey Siri, what’s the weather?” — your phone understood your voice
- ▶ **Auto-captions:** YouTube, TikTok, and Instagram auto-generate subtitles
- ▶ **Music recognition:** Shazam identifies a song from a 5-second clip
- ▶ **Voice messages:** WhatsApp can transcribe voice notes to text
- ▶ **Video calls:** Google Meet does live captions during meetings
- ▶ All of these need AI that can *understand sound*
- ▶ And the same *Transformer architecture* from Day 17 powers most of them!

- ▶ **Text** is nice: it comes as clear, separate tokens (words)
- ▶ **Images** are nice: they come as a grid of pixel values
- ▶ **Audio?** It's a *messy continuous stream*:
 - Where does one word end and the next begin? (There's no "space" in speech!)
 - The same word sounds different from different speakers
 - Background noise, echoes, overlapping voices...
 - 16,000 numbers *per second* — that's a lot of data
- ▶ **Today we'll learn:**
 - How to convert this messy stream into something a neural network can work with
 - How modern models (Whisper, HuBERT) handle all that complexity
 - It turns out: with the right preprocessing, audio isn't so different from images!

Part 1: Audio Fundamentals

- ▶ What is sound? Vibrations, pitch, volume
- ▶ How computers record sound (sampling)
- ▶ Spectrograms: turning sound into pictures
- ▶ Mel scale: matching human hearing
- ▶ History: from terrible to amazing

Part 2: Modern Audio Models

- ▶ Self-supervised learning for audio
- ▶ Wav2Vec 2.0 and HuBERT
- ▶ Whisper: speech recognition that just works
- ▶ Multimodal: LLMs that can hear
- ▶ What you can build with all this

Key insight: Sound is just another kind of data.

Convert it to numbers → use the same deep learning tools we already know.

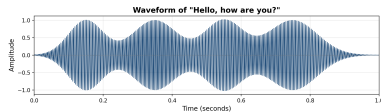
- ▶ Try this: put your hand on your throat and say “Aaaaah”
- ▶ Feel that vibration? That’s exactly what sound is: *things vibrating*
- ▶ When you speak, your vocal cords vibrate back and forth very fast
- ▶ Those vibrations push air molecules, which push the next ones, which push the next...
- ▶ Like a chain of dominoes — the vibration *travels* through the air
- ▶ When it reaches someone’s ear, their eardrum vibrates too — and they hear you!
- ▶ **Sound travels at ~ 343 m/s in air** (about 1,235 km/h)
- ▶ That’s why you see lightning before hearing thunder — light is much faster

Pitch and Volume: The Two Things You Can Control

- ▶ When you hum, you can make *two* obvious changes:
- ▶ **Pitch** (high or low) = how *fast* the vibration repeats
 - A squeaky mouse voice: vibrations repeat very fast (high pitch)
 - A deep bass drum: vibrations repeat slowly (low pitch)
 - We measure this in **Hertz (Hz)** = vibrations per second
 - A guitar string playing middle A vibrates **440 times per second** = 440 Hz
- ▶ **Volume** (loud or quiet) = how *big* each vibration is
 - Shouting = big vibrations = loud
 - Whispering = tiny vibrations = quiet
- ▶ **Human ears can hear:** 20 Hz (very deep rumble) to 20,000 Hz (very high squeal)
- ▶ Most speech lives between 85–4,000 Hz

What Does Sound Look Like?

- ▶ If you record someone saying “Hello, how are you?” and plot the vibration over time, you get a *waveform*
- ▶ **How to read it:**
 - **X-axis:** time (left = beginning, right = end)
 - **Y-axis:** how far the air is pushed (amplitude)
 - Tall peaks = loud moments
 - Flat parts = silence or quiet sounds
- ▶ Notice the bumps — each bump is roughly one *syllable*!
- ▶ “Hel” – “lo” – “how” – “are” – “you”

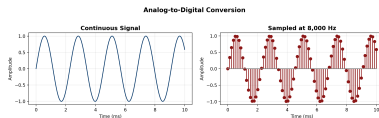


Our running example: the waveform of “Hello, how are you?”

Why Voice, Piano, and Guitar Sound Different

- ▶ Here's something interesting: a piano and a guitar can play the *same note* (same pitch, same volume) — but they sound completely different. Why?
- ▶ Because real sounds aren't simple — they're a *mix of many vibrations at once*
- ▶ Think of it like colors:
 - Pure red light = one single color (like a single vibration frequency)
 - But most things you see are *mixtures*: the brown of wood is red + orange + a bit of green
- ▶ Sound works the same way:
 - A tuning fork: one pure vibration (boring beeeep)
 - Your voice: *hundreds* of vibrations layered together — that's what makes it sound like *you*
 - A piano: a different mix of vibrations — same note, different “recipe”

- ▶ In real life, sound is *continuous* — it exists at every tiny instant of time
- ▶ But computers can only store individual numbers, not continuous waves
- ▶ **Solution: take snapshots!**
 - Measure the wave's height many times per second
 - Each measurement = one number
 - This is called *sampling*
- ▶ **How often?** That's the *sampling rate*:
 - Phone calls: 8,000 times/sec (sounds a bit muffled)
 - Siri/Whisper: **16,000 times/sec**
 - Music (Spotify): 44,100 times/sec



Left: the real continuous sound.
Right: the computer's version — just dots (samples).

How Many Snapshots Is Enough?

- ▶ If you sample too *slowly*, you miss important details and the sound gets distorted
- ▶ Like taking a photo of a spinning wheel with a slow camera — it looks blurry or weird
- ▶ There's a simple rule called the *Nyquist Rule*:

$$\text{Sampling rate} \geq 2 \times \text{highest pitch in the sound}$$

- ▶ **In plain English:** sample at least twice as fast as the fastest vibration

- ▶ **Examples:**

- Speech goes up to $\sim 8,000$ Hz $\rightarrow$ need at least 16,000 samples/sec ✓
- Music goes up to $\sim 20,000$ Hz $\rightarrow$ need at least 40,000 samples/sec ✓ (CDs use 44,100)

- ▶ **What if you don't follow this rule?**

- ▶ After sampling, sound becomes something very familiar to a programmer:
[0.02, 0.05, 0.11, 0.18, 0.23, 0.19, 0.12, 0.04, -0.03, ...]
- ▶ That's it! A *list of numbers*. Each number = how pushed the air was at that moment.
- ▶ **How big is this list?**
 - 1 second at 16,000 Hz = **16,000 numbers**
 - 10 seconds = **160,000 numbers**
 - A 3-minute song at CD quality = **~8 million numbers**
- ▶ **The problem for AI:** these lists are really, *really* long
- ▶ And the raw numbers don't directly tell us *what sounds are present*
- ▶ It's like reading a novel one letter at a time — technically everything is there, but you can't see the big picture
- ▶ We need a *smarter way to look at sound* — spectrograms

- ▶ When someone plays a chord on a guitar, you hear *multiple notes at once*
- ▶ Your brain can pick out the individual notes — “that’s a C and an E and a G”
- ▶ But look at the waveform and all you see is a messy squiggle
- ▶ The individual notes are *hidden inside* that squiggle, all mixed together
- ▶ **Same with speech:**
 - The vowel “ah” has certain pitches mixed together
 - The consonant “s” has completely different pitches (high, hissy ones)
 - In the raw waveform, they’re all blended — we can’t tell them apart
- ▶ **What we want:** A way to “un-mix” the sound and see which pitches are present at each moment
- ▶ This is like having X-ray vision for sound!

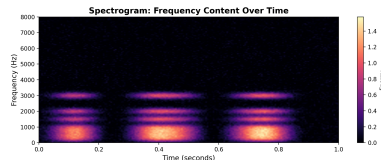
- ▶ Imagine you're given a smoothie. Can you figure out the ingredients just by looking at it?
- ▶ Hard! But a lab could analyze it and say: "40% banana, 30% strawberry, 20% yogurt, 10% honey"
- ▶ Sound works the same way. There's a math tool called the *Fourier Transform* that can "taste" a chunk of sound and tell you:
 - "There's a strong vibration at 200 Hz" (that's the voice's base tone)
 - "There's some energy at 600 Hz and 1200 Hz" (those are the overtones)
 - "Almost nothing at 5000 Hz" (no hissy sounds right now)
- ▶ **Input:** a short chunk of audio (the "smoothie")
- ▶ **Output:** a list of which frequencies are present and how strong each one is (the "ingredients")
- ▶ You don't need to understand the math — just the idea: *it splits sound into*

- ▶ Speech *changes over time* — the “H” in “Hello” has different frequencies than the “o”
- ▶ So we can't just analyze the whole clip at once — we need to check *moment by moment*
- ▶ **Here's what we do:**
 1. Take a tiny window of audio (about 25 milliseconds — way less than a blink)
 2. Use the Fourier Transform to find which frequencies are in that window
 3. Write down the result as one *vertical stripe* of data
 4. Slide the window forward a little (10 ms) and repeat
 5. Keep going until you've covered the whole audio clip
- ▶ Stack all those vertical stripes side by side and you get a *spectrogram*!
- ▶ It's basically a **heat map** showing:

- **Left to right:** time

What a Spectrogram Looks Like

- ▶ Each sound has a *visual fingerprint* in the spectrogram:
- ▶ **Vowels** (“ah”, “ee”, “oo”): bright horizontal bands — the voice holds steady frequencies
- ▶ **“S” and “sh” sounds**: bright fuzzy cloud at the *top* — these are high-pitched hissy noises
- ▶ **“P”, “t”, “k” sounds**: short vertical bursts — a quick pop of energy
- ▶ **Silence**: dark/black — nothing happening
- ▶ Linguists have been reading spectrograms like this since the 1940s!
- ▶ With practice, you can literally *read words* from a spectrogram



Spectrogram of “Hello, how are you?” — each bright region is a different sound.

Why Spectrograms Are a Game-Changer for AI

- ▶ **Remember:** raw audio is a huge list of numbers (16,000 per second)
- ▶ A spectrogram compresses this into a much smaller *2D grid*
- ▶ For a 1-second clip:
 - Raw audio: **16,000 numbers** (hard for a neural network)
 - Spectrogram: about $257 \times 98 \approx 25,000$ **numbers** in a 2D grid
- ▶ Wait — 25,000 is *more* than 16,000? Why is this better?
- ▶ Because it's an *image*! And we already know how to work with images:
 - CNNs (Day 9) can find patterns in images
 - Vision Transformers (Day 19) can process image patches
 - The same tools work on spectrograms!
- ▶ **Key insight:** By turning sound into a picture, we can use all our *image AI tools* on audio

Running Example

Our “Hello, how are you?” clip (1 second, 16,000 Hz sampling):

1. Start with 16,000 raw numbers
2. Slide a 25 ms window across the audio, stepping 10 ms each time
3. That gives us about 98 time steps (windows)
4. At each step, the Fourier Transform finds 257 frequency values
5. Result: a 257×98 grid — a small image!

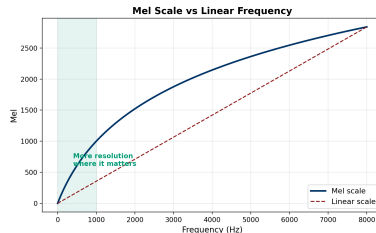
Now instead of a mysterious list of 16,000 numbers, we have a picture where we can *see* the “H”, the “e”, the “l”, the “o” as different visual patterns.

But we can do even better. The spectrogram treats all frequencies equally — but our ears don’t. Can we make it match *how humans actually hear*?

- ▶ Here's something you can test yourself:
- ▶ Play a 200 Hz tone, then a 300 Hz tone — you'll hear a *big* difference
- ▶ Now play a 5,000 Hz tone, then a 5,100 Hz tone — they sound almost *identical*
- ▶ Both pairs are 100 Hz apart — but the first pair sounds way more different!
- ▶ **Why?** Your ears care a lot about small changes at *low* pitches (where speech lives), but barely notice changes at *high* pitches
- ▶ This makes sense for survival:
 - Low pitches = voices, footsteps, animal growls → important to tell apart!
 - High pitches = birds chirping, insects buzzing → less critical to distinguish
- ▶ Our spectrogram doesn't know this — it gives *equal space* to every frequency
- ▶ Can we make it match how we actually hear?

The Mel Scale: A Ruler That Matches Your Ears

- ▶ The *mel scale* is just a different way to measure pitch
- ▶ Named after “melody” — it was designed by asking people: “When do two sounds seem equally far apart?”
- ▶ **How it works:**
 - At low pitches: mel scale *zooms in* (more detail)
 - At high pitches: mel scale *zooms out* (less detail)
- ▶ Think of it like a camera zoom:
 - Regular spectrogram = photo with uniform zoom
 - Mel spectrogram = photo that zooms into the important part (low frequencies) and shows the rest smaller



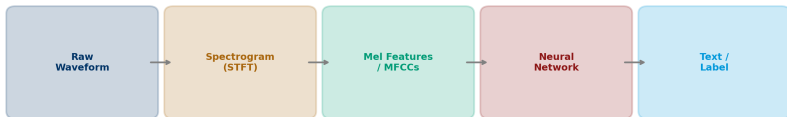
Blue curve = mel scale. It gives more space to low frequencies (where speech is) and compresses high ones.

- ▶ A *mel-spectrogram* is just a regular spectrogram passed through the mel scale
- ▶ **Steps:**
 1. Make a normal spectrogram (257 frequency rows)
 2. Group nearby frequencies into *mel bins* using the mel scale
 3. Typically use **80 bins** (Whisper uses 80, some models use 128)
 4. Apply a logarithm (because our ears perceive loudness on a log scale too)
- ▶ **Result:** 80 rows instead of 257 — smaller *and* better matched to human hearing
- ▶ **This is the standard input for modern audio AI:**
 - Whisper uses 80-bin mel-spectrograms
 - HuBERT uses mel-spectrograms
 - Almost every speech model since 2018 uses them

- ▶ For 30+ years before deep learning, people used an even more compact format: *MFCCs* (Mel-Frequency Cepstral Coefficients)
- ▶ The idea: take the mel-spectrogram (80 numbers per time step) and *compress it further* to just **13 numbers** per time step
- ▶ These 13 numbers capture the *overall shape* of the frequency content:
 - Number 1: overall loudness
 - Number 2: whether the sound is more low-pitched or high-pitched
 - Numbers 3–13: finer details of the sound's “recipe”
- ▶ **MFCCs were the gold standard for decades** — used in Siri's early versions, phone systems, and speaker recognition
- ▶ Today, deep learning models mostly use *mel-spectrograms directly* — they can learn their own way to compress the information

The Journey from Sound to AI Input

Audio Processing Pipeline



Running Example: Data Gets Smaller at Each Step

“Hello, how are you?” (1 second):

Step	What it is	Numbers
Raw audio	List of samples	16,000
Spectrogram	Frequency $\times$ time grid	25,186
Mel-spectrogram	Compressed frequency grid	7,840
MFCCs	Ultra-compact summary	1,274

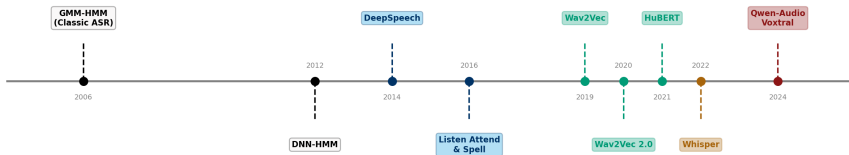
The mel-spectrogram is the sweet spot: small enough for AI, rich enough to keep all the important detail.

1. **Sound** enters the microphone as vibrations in air
2. **Sampling** converts it to a list of numbers (16,000 per second)
3. **Spectrogram** reveals which frequencies are present at each moment (a 2D image)
4. **Mel scale** adjusts the spectrogram to match how humans hear (zoom into low frequencies)
5. **Mel-spectrogram** = the standard input for modern audio AI models

Key takeaway: We turned vibrating air into a small 2D image. From here, we can use the same neural network tools we’ve been learning all camp — CNNs, Transformers, attention — on *sound*.

Next up: How did we get from “talking to a computer and it doesn’t understand” to Siri and Alexa? The history of speech recognition.

Evolution of Speech & Audio Models



- ▶ People have been trying to make computers understand speech since the **1970s**
- ▶ The dream was always the same: you talk, the computer types what you said
- ▶ But getting there was *really hard* — let's see why

- ▶ Early speech systems were like a *Rube Goldberg machine* — tons of separate parts
- ▶ **The pipeline:**
 1. Extract MFCCs (those 13 numbers per time step we just learned about)
 2. A statistical model guesses which *sound* each frame might be (“ah”, “eh”, “ss”, ...)
 3. Another model chains sounds into *words* (“ah” + “p” + “l” = “apple”)
 4. A language model picks the sentence that makes the most *sense* (“I saw the” → probably not “eye sore thee”)
- ▶ **Problems:**
 - Each part was trained separately — errors in step 2 would mess up step 3 and 4
 - Needed handmade pronunciation dictionaries (every word, spelled out phonetically)

- ▶ **2012:** Deep learning explodes — same year as AlexNet for images
- ▶ Researchers replaced the statistical sound-guesser with a *neural network*
- ▶ **What changed:**
 - Neural networks learned much better audio features than the old statistics
 - Error rates dropped **20–30%** almost overnight
 - Then RNNs and LSTMs (Day 17) brought sequence modeling — understanding that sounds *flow in order*
- ▶ **But the overall pipeline was still the same:**
 - Still needed separate models for sounds, words, and grammar
 - Still needed pronunciation dictionaries
 - Better accuracy, but still a *fragile chain* of separate parts
- ▶ **The question:** Can we make *one* model that does *everything*?

- ▶ **The breakthrough:** One single model. Audio goes in, text comes out. No pipeline.
- ▶ **Key models:**
 - *Wav2Vec 2.0* (2020): Learns from raw audio without labels (like BERT for sound)
 - *HuBERT* (2021): Same idea, even better results
 - *Whisper* (2022): Trained on 680,000 hours of audio — works in 99 languages
- ▶ **Why does this work so much better?**
 - No more error cascading between separate parts
 - The model figures out sounds, words, and grammar *all at once*
 - And the Transformer (Day 17) is the engine behind all of them!
- ▶ **Today:** Whisper can transcribe your speech nearly as well as a human can
- ▶ We went from “please speak clearly into the microphone” to “just talk”

	Old approach	Modern approach
Images (Days 9–10)	Hand-designed filters	CNNs learn their own filters
Text (Days 17–18)	Rules + dictionaries	Transformers learn from data
Audio (today)	Pipeline of separate models	End-to-end Transformers

- ▶ In every domain, the story is the same:
 1. Start with hand-designed features and pipelines
 2. Replace pieces with neural networks
 3. Eventually, one big model replaces the whole pipeline
- ▶ The *Transformer architecture* from Day 17 keeps showing up as the final answer
- ▶ Text, images, and now audio — it's the *universal AI building block*

The Big Problem: Who's Going to Label All That Audio?

- ▶ To train a speech recognizer, you need *pairs*: an audio clip + the correct text
- ▶ Someone has to *listen* to each clip and *type out* exactly what was said
- ▶ How expensive is this?
 - A human transcriber can do about 15 minutes of audio per hour of work
 - That costs \$1–3 per minute of audio
 - 1,000 hours of labeled data $\approx$ \$60,000–180,000!
- ▶ And most languages barely have any labeled speech data at all
- ▶ English has thousands of hours. But Swahili? Yoruba? Many Arabic dialects? Almost nothing.
- ▶ **Meanwhile:** there are *millions of hours* of unlabeled audio on YouTube, podcasts, radio — all free, but no one typed out what was said
- ▶ Can we somehow learn from all that *unlabeled* audio?

- ▶ Remember *BERT* from Day 18?
 - Take a sentence: “The cat sat on the [MASK]”
 - Hide a word and ask the model to guess it
 - No labels needed — the text *is* the label!
- ▶ We can do the **exact same thing with audio!**
 - Take an audio clip and *mask out* (hide) random chunks
 - Ask the model: “What was the person saying during the hidden parts?”
 - The model learns speech patterns without anyone typing out transcriptions
- ▶ This is called *self-supervised learning*:
 - “Self” because the data provides its own training signal
 - No human labels required
 - Train on millions of hours of free audio from the internet

- ▶ **Wav2Vec 2.0** (Meta, 2020): the first big self-supervised audio model
- ▶ **How it works:**
 1. Feed in raw audio (just the list of numbers)
 2. A small CNN chops it into short feature chunks
 3. *Mask* (hide) some of those chunks
 4. A Transformer tries to *predict* what the hidden chunks should be
- ▶ By guessing the hidden parts over and over across millions of hours, the model learns:
 - What speech sounds like (the building blocks: “ah”, “ee”, “ss”, ...)
 - How sounds flow together (“th” is often followed by “e” or “a”)
 - What different speakers and accents sound like
- ▶ **Mind-blowing result:** With just *10 minutes* of labeled data after pre-training, Wav2Vec 2.0 matched systems trained on *960 hours* of labels!

- ▶ **HuBERT** (Meta, 2021): “Hidden-Unit BERT” — an improved version
- ▶ **What’s different?**
 - Before training, HuBERT groups similar-sounding audio chunks into categories (using a simple clustering algorithm)
 - These categories are like *automatically discovered sound units* — the model figures out “ah”, “ee”, “ss” on its own, with no human help!
 - Then it trains by masking audio and predicting which category the hidden chunk belongs to
- ▶ **HuBERT is really versatile — it can be fine-tuned for:**
 - *Speech recognition*: what did they say?
 - *Speaker identification*: who is speaking?
 - *Emotion detection*: are they happy, angry, or sad?
 - *Language identification*: what language is this?

	BERT (Text, Day 18)	HuBERT (Audio)
Input	Words/tokens	Audio frames
Pre-training trick	Mask random words	Mask random audio chunks
What it predicts	The missing word	The missing sound category
Pre-trained on	Billions of words (free)	60,000+ hours of audio (free)
Fine-tune for	Text tasks (QA, classification)	Audio tasks (ASR, speaker ID)
Labels needed	Very few	Very few (even 10 min works!)

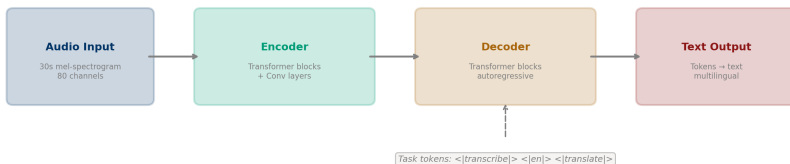
- ▶ **The pattern:** Self-supervised pre-training works across all data types
- ▶ Learn general patterns from *tons of free data*, then specialize with *a little labeled data*
- ▶ This is huge for *low-resource languages*:
 - Pre-train on any audio in any language (millions of hours available)
 - Fine-tune with just a few hours of labeled speech in the target language

But What If We Just Had... More Labels?

- ▶ Self-supervised learning is clever: learn from free unlabeled data, fine-tune with a little labeled data
- ▶ But what if someone took a *completely different approach*?
- ▶ What if instead of being clever about labels, you just... *got a LOT more labels*?
- ▶ The internet has millions of YouTube videos with subtitles
- ▶ Podcasts come with transcripts
- ▶ Audiobooks have the original text
- ▶ What if you gathered **680,000 hours** of audio with (noisy) text labels from the internet and just trained a massive model on all of it?
- ▶ That's exactly what *Whisper* did — and it worked incredibly well
- ▶ Let's see how →

- ▶ HuBERT was clever: learn from free unlabeled audio, then fine-tune with a little labeled data
- ▶ **Whisper** (OpenAI, 2022) took a completely different approach: forget being clever — just use *way more data*
- ▶ **Training data:** 680,000 hours of audio with text from the internet
 - YouTube videos with subtitles
 - Podcasts that had transcripts
 - Audiobooks with the original text
 - In **99 different languages**
- ▶ 680,000 hours = listening 24/7 for **77 years**
- ▶ The subtitles weren't perfect (some were auto-generated) — but with that much data, the model learned to handle the noise

Whisper Architecture (Simplified)



- ▶ It's the **encoder-decoder Transformer** from Day 17 — nothing new!
- ▶ **Input**: Take 30 seconds of audio → convert to a mel-spectrogram (80 frequency bins)
- ▶ **Encoder**: Transformer blocks read the mel-spectrogram and understand the speech

- ▶ Let's trace what happens when you give Whisper our running example:
 1. You record "Hello, how are you?" and save it as `hello.wav`
 2. Whisper loads the audio and converts it to a mel-spectrogram (the 2D frequency image we learned about)
 3. The *encoder* (Transformer blocks) reads this image and builds a deep understanding of the speech
 4. The *decoder* starts generating: "Hello" → "," → "how" → "are" → "you" → "?"
 5. Done! You get back the text: "Hello, how are you?"
- ▶ The decoder generates text *one token at a time*, feeding each word back in to predict the next — exactly like GPT (Day 18)
- ▶ The only difference: instead of getting context from previous text, it gets context from *audio*

- ▶ Whisper isn't just a speech-to-text machine — it can do *multiple tasks*!
- ▶ You tell it what to do using special **task tokens**:
- ▶ **Transcribe**: Write down speech in the same language
 - Arabic speech + `<|transcribe|><|ar|>` → Arabic text
 - English speech + `<|transcribe|><|en|>` → English text
- ▶ **Translate**: Convert any language to English text
 - Arabic speech + `<|translate|>` → English text (!)
 - Japanese speech + `<|translate|>` → English text
- ▶ **Auto-detect language**: Don't specify a language tag and Whisper figures it out
- ▶ All of this from *one model*. No separate translation system, no separate language detector — it's all built in.

Whisper Comes in Different Sizes

Model	Parameters	Accuracy	Speed
Tiny	39M	Good	Very fast (real-time on phone)
Base	74M	Better	Fast
Small	244M	Great	Moderate
Medium	769M	Excellent	Needs good GPU
Large-v3	1.55B	Near-human	Needs powerful GPU

- ▶ **WER** = Word Error Rate (what % of words it gets wrong)
- ▶ Whisper Large-v3 gets about **2%** of English words wrong — that's almost as good as a human transcriber
- ▶ Even Tiny (39M params) beats many older speech systems that were 10× bigger
- ▶ **Pick based on your needs:**
 - Live captions on your phone? Use Tiny or Base
 - Transcribing a podcast for a blog post? Use Medium or Large

	HuBERT	Whisper
How it learned	Filling in blanks (self-supervised)	From subtitles (supervised)
Labeled data needed	A little (for fine-tuning)	None (already trained)
Best at	Many audio tasks	Speech-to-text
Languages	One at a time (fine-tune each)	99 built in
Who is speaking?	<i>Great at this</i>	Not really
Translation?	<i>X</i>	<i>Built-in</i>
Works out of the box?	Needs fine-tuning first	Yes!

- ▶ **Want speech-to-text?** → Whisper
- ▶ **Want to identify who's speaking or detect emotion?** → HuBERT
- ▶ **They complement each other** — many real systems use both!

Whisper in Python: It's This Easy

```
import whisper

# Load the model (downloads ~140MB for "base")
model = whisper.load_model("base")

# Transcribe any audio file
result = model.transcribe("hello.wav")

print(result["text"])
# "Hello, how are you?"

print(result["language"])
# "en"
```

- ▶ **5 lines of code.** That's the entire speech recognition pipeline.
- ▶ No mel-spectrogram code, no decoder setup, no language model — Whisper handles all of it internally
- ▶ Works with WAV, MP3, FLAC, M4A — any audio format

Running Example

```
# Transcribe Arabic speech to Arabic text
result = model.transcribe("arabic_speech.wav",
                           language="ar")

print(result["text"])
# Arabic text output

# Translate Arabic speech to English text!
result = model.transcribe("arabic_speech.wav",
                           task="translate")

print(result["text"])
# English translation of what was said
```

- ▶ Just change task="translate" and Whisper converts any language to English
- ▶ No separate translation model needed
- ▶ This works for **99 languages** including Arabic, Chinese, Japanese, French, Spanish, ...

- ▶ Whisper is great at one thing: converting speech → text
- ▶ But what if you want to actually *understand* the audio, not just transcribe it?
- ▶ Imagine sending an audio clip to a chatbot and asking:
 - “Is this person angry or calm?”
 - “What instruments are playing in this song?”
 - “Summarize what the speaker is saying”
 - “Translate this conversation and tell me the key points”
- ▶ This needs *two skills at once*:
 - Understanding audio (what’s in the sound?)
 - Language reasoning (answering questions, summarizing, explaining)
- ▶ **Solution:** Give an LLM the ability to “hear” — *multimodal audio models*

- ▶ Remember from Day 19 how we gave LLMs the ability to “see” images?
 - ViT converts an image $\rightarrow$ embedding vectors
 - Feed those vectors to the LLM alongside text
 - Now the LLM can reason about images!
- ▶ **Audio works the exact same way:**
 1. Take a pre-trained audio encoder (like Whisper’s encoder or HuBERT)
 2. It converts audio $\rightarrow$ embedding vectors
 3. A small *adapter layer* translates these into the LLM’s language
 4. Feed them to the LLM alongside the text question
 5. The LLM now “hears” the audio and can answer questions about it
- ▶ **The beautiful thing:** We reuse pre-trained models for both parts
- ▶ Audio encoder already understands sound + LLM already understands language $\rightarrow$ together they understand *audio + language*

- ▶ **Qwen-Audio** (Alibaba, 2023): An LLM that can hear
- ▶ Uses a Whisper-style encoder + the Qwen LLM (7 billion parameters)
- ▶ **What it can do — all with one model:**
 - *Speech recognition*: “What did the person say?” → transcription
 - *Audio captioning*: “Describe this sound” → “A dog barking with traffic in the background”
 - *Music understanding*: “What genre is this?” → “Upbeat pop, 120 BPM”
 - *Sound detection*: “What sounds do you hear?” → “Glass breaking at 2.3 seconds”
 - *Audio Q&A*: Play a lecture, ask “What was the main argument?”
- ▶ Each of these used to need a *separate specialized model*
- ▶ Now one model handles them all — just by changing your question

- ▶ **Voxtral** (Mistral AI, 2024): Another LLM with ears
 - Can process audio up to *30 minutes long*
 - Can handle multiple audio files in one conversation
 - Great for: meeting summaries, podcast analysis, customer service calls
- ▶ **The big trend:** LLMs are becoming *multimodal* — they can handle many types of input:

Model	Can read text	Can also...
GPT-4 (2023)	✓	See images
GPT-4o (2024)	✓	See images + hear audio
Gemini (2024)	✓	See images + hear audio + watch video
Qwen-Audio	✓	Hear audio
Voxtral	✓	Hear audio

- ▶ The future: one AI that can *read, see, hear, and speak* — all at once

- ▶ Look at what we've seen across this whole camp:
 1. **Text:** tokens $\rightarrow$ embeddings $\rightarrow$ Transformer $\rightarrow$ output
 2. **Images:** patches $\rightarrow$ embeddings $\rightarrow$ Transformer $\rightarrow$ output
 3. **Audio:** mel frames $\rightarrow$ embeddings $\rightarrow$ Transformer $\rightarrow$ output
- ▶ It's the *same architecture every time!*
- ▶ The only thing that changes is how you convert the raw input into embeddings
- ▶ **This is the key insight of modern AI:**
 - Text, images, audio, video — they're all just different types of *sequences*
 - Convert them to embeddings and the Transformer handles the rest
 - One architecture to rule them all

Speech Tasks

- ▶ **Transcription:** Speech → text (Siri, Google, Whisper)
- ▶ **Speaker ID:** “Who is talking?” (phone unlock by voice)
- ▶ **Emotion detection:** “Are they angry or happy?” (call centers)
- ▶ **Language ID:** “What language is this?” (auto-translate)
- ▶ **Noise removal:** Clean up background noise in calls

Non-Speech Audio Tasks

- ▶ **Sound classification:** Dog bark? Car horn? Gunshot? (security cameras)
- ▶ **Music genre:** Rock? Jazz? Pop? (Spotify recommendations)
- ▶ **Music recognition:** “What song is this?” (Shazam)
- ▶ **Audio captioning:** “Describe what you hear”
- ▶ **Medical:** Cough analysis, heart sounds

The amazing part: A single pre-trained model can be adapted for *all* of these!

- ▶ Remember **transfer learning** from Day 10?
 - Take a model pre-trained on ImageNet (millions of images)
 - Swap out the last layer for your task (cats vs dogs, medical scans, etc.)
 - Fine-tune with a small dataset — works great!
- ▶ **Audio transfer learning works exactly the same way:**
 1. Start with a pre-trained audio model (HuBERT or Whisper's encoder)
 2. These models already “understand” speech, sounds, and audio patterns
 3. **Freeze** the pre-trained weights (don't change what it already learned)
 4. Add a small new layer for your specific task
 5. Fine-tune on your (small) labeled dataset
- ▶ **Example:** Want to detect if a caller is angry?
 - Take HuBERT + add one layer that outputs [happy, sad, angry, neutral]

- Let's trace our running example through the *entire pipeline* one last time:

Running Example: The Complete Journey

1. Someone speaks → vocal cords vibrate → air molecules push each other
2. **Microphone** converts vibrations → electrical signal
3. **Sampling** at 16,000 Hz → a list of 16,000 numbers per second
4. **Spectrogram**: slide a window across, find frequencies → 2D heat map
5. **Mel scale**: zoom into low frequencies that matter most → 80-bin mel-spectrogram
6. **Whisper encoder** (Transformer): reads the mel-spectrogram, understands the speech
7. **Whisper decoder** (Transformer): generates text one word at a time
8. **Output**: "Hello, how are you?"

From vibrating air to text — all in *under 1 second*.

- ▶ After today, you have the conceptual toolkit to understand — and build on — real audio AI systems:
- ▶ **With Whisper (5 lines of Python):**
 - Auto-generate subtitles for any video
 - Build a voice-controlled app
 - Transcribe meetings or lectures automatically
 - Translate speech across languages
- ▶ **With HuBERT + fine-tuning:**
 - Build a speaker identification system (who said what in a meeting?)
 - Detect emotions in customer service calls
 - Identify languages automatically
- ▶ **With multimodal models (Qwen-Audio):**

Part 1: Audio Fundamentals

- ▶ Sound = vibrations with pitch and volume
- ▶ Computers record by sampling (16,000 times/sec for speech)
- ▶ Nyquist rule: $\text{sample} \geq 2 \times$ highest frequency
- ▶ Spectrograms: X-ray vision for sound
- ▶ Mel scale: zoom into frequencies humans care about
- ▶ MFCCs: ultra-compact sound summary (classic)

Part 2: Modern Models

- ▶ Self-supervised: mask audio chunks, predict them (like BERT)
- ▶ HuBERT: discovers sound units automatically
- ▶ Whisper: 680K hours of data, 99 languages, works out of the box
- ▶ Multimodal: LLMs + audio encoder = AI that can hear and reason
- ▶ Transfer learning: one model $\rightarrow$ many audio tasks

The big takeaway: Sound $\rightarrow$ numbers $\rightarrow$ mel-spectrogram (image) $\rightarrow$ Transformer $\rightarrow$ understanding.
Same deep learning tools, new type of data.

Concept	Key Fact
Sampling rate	Snapshots per second (16,000 Hz for speech AI)
Nyquist rule	Sample at $\geq 2\times$ the highest frequency
Spectrogram	2D heat map: time $\times$ frequency (from STFT)
Mel scale	Zooms into low frequencies (where speech lives)
Mel-spectrogram	80 mel bins — the standard AI input
MFCCs	13 numbers per frame — classic compact features
Self-supervised	Mask & predict audio (no labels needed)
Wav2Vec 2.0	BERT-style pre-training on raw audio
HuBERT	Discovers sound categories, then mask & predict
Whisper	680K hrs supervised, encoder-decoder Transformer
Whisper sizes	Tiny (39M) to Large (1.55B), 99 languages
Multimodal audio	Audio encoder + LLM = AI that hears and reasons

The Transformer: One Architecture for Everything

Domain	Input	Convert to	Model
Text (Day 17–18)	Words	Token embeddings	BERT, GPT
Images (Day 19)	Pixels	Patch embeddings	ViT
Audio (Today)	Sound waves	Mel-spectrogram	Whisper, HuBERT
<i>All combined</i>	<i>Everything</i>	<i>Embeddings</i>	<i>GPT-4o, Gemini</i>

- ▶ The pattern is always the same:
 1. Convert your data into a sequence of vectors (embeddings)
 2. Feed them into a Transformer
 3. The Transformer does the rest
- ▶ This is the most important idea in modern AI:
- ▶ **One architecture, any data type, any task.**

What's next? These same ideas extend to video, 3D data, scientific measurements, and beyond.

Credits

KAUST Academy

King Abdullah University of Science and Technology